

Report of Classification Models

Yansong He
2830791284@qq.com

Abstract

本报告探讨了三种不同的分类模型：决策树、AdaBoost 结合决策树及支持向量机（SVM）使用核方法的性能。我们利用一个包含 1500 个样本的合成 3D 数据集进行模型评估，该数据集被分为两个类别。我们依据模型的准确率、精确率、召回率和 F1-score 等指标对模型的有效性进行了评估。结果显示，结合弱学习器的 AdaBoost 模型表现最佳，达到了 98.90% 的准确率。SVM 模型在使用高斯径向基核时，其准确率紧随其后，达到了 98.70%。而单独的决策树模型则表现相对较弱，准确率为 96.40%。这些结果突显了不同模型在分类任务上的优缺点，表明集成方法如 AdaBoost 在提升分类性能方面具有显著优势。

Introduction

分类是机器学习和数据分析中的一个基本任务，是众多应用领域的基础，包括图像识别、金融分析和医疗诊断等。本文旨在探讨三种广泛应用的分类算法：决策树、AdaBoost 结合决策树和支持向量机（SVM）使用核方法。

决策树是一种直观且易于理解的监督学习算法，通过树状结构对数据进行分类。在本实验中，我们对决策树的深度进行了调优，以提高模型性能。然而，决策树往往容易受到过拟合的影响，特别是在处理复杂数据集时。AdaBoost（自适应提升）是一种集成学习方法，通过将多个弱分类器（如小型决策树）组合成一个强大的预测器，旨在提高分类精度。该算法的核心在于逐步调整训练样本的权重，使得每个弱学习器更加关注于之前模型的错误，从而实现更好的泛化能力。支持向量机（SVM）是一种强大的分类器，特别是在高维特征空间中表现优异。其主要通过最大化类别间的边际来实现分类，结合核方法，可以有效处理非线性分类问题。然而，SVM 的性能高度依赖于超参数的选择，如正则化参数 C 和核函数的类型。

在随后的部分中，我们将详细介绍所采用的模型及其训练过程，并分析每个模型在合成数据集上的分类表现。通过对比结果，我们希望能对不同分类模型在实际应用中的有效性提供有用的见解。

M1: Decision Trees

(1) 定义

- 一种基于树结构的监督学习算法
- 通过树状结构进行数据分类和预测
- 可用于分类和回归问题

(2) 决策树的基本结构

- 根节点**：包含整个数据集
- 内部节点**：特征属性测试点
- 分支**：测试结果
- 叶子节点**：类别标签

(3) 信息熵 (Entropy)

$$H(X) = \log \left(\frac{1}{P(X)} \right) = -\log P(X)$$

熵量化信息或信号中固有的信息量或惊喜。熵越高，信息越不可预测，反之亦然。香农的解遵循信息函数H (X)的基本性质，通过满足以下性质来衡量信息量：

- H(X) 是反单调的——事件x概率的增加和减少分别产生信息的减少和增加。
- $H(X) \geq 0$ ，信息是一个非负的量
- $H(1) = 0$ ，总是发生的事件不会传达信息
- $H(X_1, X_2) = H(X_1) + H(X_2)$ ，由独立事件引起的信息是可加的

$$Entropy(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

- 度量数据集的纯度（需计算每一个事件的概率和信息熵，再作和）
- 熵越小，数据越纯

(4) 信息增益 (Information Gain)

$$IG(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

- 衡量分裂后信息减少量（将数据集根据 A 分裂为好几个数据集，则可分别计算每个数据集的信息熵，最后计算信息增益）
- 选择信息增益最大的特征作为分裂节点（实现数据分类）

(5) 基尼系数 (Gini Index)

$$Gini(S) = 1 - \sum_{j=1}^c p(j|S)^2$$

- 衡量数据集的不确定性
- 基尼系数越小，数据越纯

M2: AdaBoost + Decision Trees

(1) 定义

AdaBoost（自适应提升）是一种迭代的集成学习方法，核心特点是通过顺序方式构建模型，每个新模型都专注于纠正前一个模型的错误。

AdaBoost背后的核心原理是在反复修改的数据版本上拟合一系列**弱学习器**（即仅比随机猜测略好的模型，例如小型决策树）。然后通过加权多数投票（或求和）组合所有预测，以产生最终预测。

(2) 算法详细步骤

- 权重的初始化：所有训练样本权重相等，其中 T 是迭代的总数：

$$w_i = \frac{1}{N}$$

- 弱学习器的加权错误：

$$err_t = \frac{\sum_{i=1}^N w_i \cdot I(y_i \neq h_t(x_i))}{\sum_{i=1}^N w_i}$$

- 分类器的权重：

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - err_t}{err_t} \right)$$

- 更新训练样本的权重：

$$w_i^{(t+1)} = w_i^{(t)} \cdot \exp(-\alpha_t y_i h_t(x_i))$$

- 最终权重需要归一化。这可以通过将每个权重除以更新后的权重总和来实现，例如：

$$w_i^{(t+1)} := w_i^{(t)} / \sum_{i=1}^N w_i^{(t+1)}$$

- 输出最终模型：（二分类）

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$$

M3: SVM + Kernel Methods

感知器算法的局限性在于仅找到一个可分超平面、对噪声敏感、没有最优性保证。而最大化支持向量机（SVM）中的边距涉及找到分隔每侧最近数据点距离最大的类的超平面。机器学习的这个过程可以理解为一个受约束的最佳化问题——一个研究得很好的数学领域。

(1) SVM 实现步骤

边距定义为超平面与数据集中最近点（支持向量）之间的距离。对于正确分类的数据集，所有点都 x 满足：

$$y^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1, \quad \forall i$$

裕度 M 由以下公式给出：

$$M = \frac{2}{\|\mathbf{w}\|}$$

因此，最大化裕度 M 相当于最小化 $\|\mathbf{w}\|$ ，为了数学方便，**目标函数**变为

$$\min \left(\frac{1}{2} \|\mathbf{w}\|^2 \right)$$

(2) 优化问题

原始机器学习（二分类）问题可以重新表述为最佳化问题：

$$\min \left(\frac{1}{2} \|\mathbf{w}\|^2 \right) \quad s.t.: \quad y^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1, \quad i = 1, \dots, N$$

最小值不能直接通过求解目标函数的导数来找到。由于有约束，我们需要首先采取目标函数的朗格朗日来求解最小值。

(3) 拉格朗日

为了解决这个约束最佳化问题，我们为每个约束引入拉格朗日乘数

$$\alpha \geq 0$$

并构造拉格朗日：

$$\mathcal{L}(\mathbf{w}, b, \alpha) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^N \alpha_i [y^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) - 1]$$

(4) 双重问题

微分拉格朗日并将导数设置为零：

$$\nabla_{\mathbf{w}} \mathcal{L} = 0 \quad \Rightarrow \quad \mathbf{w} = \sum_{i=1}^n \alpha_i y^{(i)} \mathbf{x}^{(i)}$$

替换 w 并得到对偶形式 L(α)，通过最小化它，我们可以得到：

$$\begin{aligned} \max_{\alpha} (\mathcal{L}) = \max_{\alpha} \left(\sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y^{(i)} y^{(j)} \mathbf{x}^{(i)} \cdot \mathbf{x}^{(j)} \right) \\ s.t.: \quad \alpha_i \geq 0, \quad \sum_{i=1}^N \alpha_i y^{(i)} = 0 \end{aligned}$$

原来的约束最佳化问题最后变成二次规划问题，找到最优 αi 来最小化 L(α)。

(5) 解决双重问题

计算w和b：

$$\mathbf{w} = \sum_{i=1}^N \alpha_i y^{(i)} \mathbf{x}^{(i)}$$

$$\frac{\partial \mathcal{L}}{\partial b} = 0 \quad \Rightarrow \quad \sum_{i=1}^n \alpha_i y^{(i)} = 0$$

对于任何支持向量 xs：

$$y^{(s)}(\mathbf{w}^T \mathbf{x}^{(s)} + b) = 1$$

顺序最小优化（SMO）算法的工作原理是在每一步选择两个拉格朗日乘法器，并在保持所有其他乘法器固定的情况下找到这些乘法器的最优值。这两个乘法器是使用启发式选择的，并进行优化，使它们符合SVM最佳化问题的约束。

(6) 软边距与回归

我们只是在调用的目标函数中使用了一个额外的惩罚因子 ξ_j 。这个因子是从相应的支持超平面到另一个类的数据点超过的距离。

因此，如果数据点在边界内（支持超平面），惩罚因子为0。否则，如果数据点在另一侧，则该因子等于数据点与支持超平面之间的距离。因此，值是一个非负数。

$$\min_{\mathbf{w}, b, \xi} \left(\frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \right)$$

受分类边际限制：

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \forall i$$

$$\xi_i \geq 0, \quad \forall i$$

内核方法是一类用于支持向量机（SVM）及其他机器学习算法的技术，它通过将数据从原始特征空间映射到更高维的特征空间，从而帮助处理非线性问题。内核方法的核心思想是通过计算样本之间的相似性，借助核函数避免显式地进行特征转换。

(1) 原理

- 特征映射：

- 内核方法假设存在一个映射函数 Φ ，将原始数据点从特征空间 R^n 映射到高维特征空间 F 。然而，实际的映射形式通常是未知的。

- 核函数：

- 核函数

$$K(\mathbf{x}, \mathbf{y}) = \langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle$$

用于计算两个输入样本 $\mathbf{x}$ 和 $\mathbf{y}$ 在高维空间中的内积。通过选择合适的核函数，可以在隐式的情况下获得高维特征，而无需知道具体的映射形式 Φ 。

- 优化问题：

- 支持向量机的优化目标是找到一个超平面来最大化类间的间隔。在引入核函数后，优化问题中的内积计算可以被替换为核函数计算，从而简化决策过程并允许高维处理。

(2) 常用的核函数

- 线性核 (Linear Kernel)

- 公式：

$$K(\mathbf{x}, \mathbf{y}) = \mathbf{x}^\top \mathbf{y}$$

- 特点：

- 适用于线性可分的数据。
- 计算简单且效率高。

- **多项式核 (Polynomial Kernel)**

- **公式:**

$$K(\mathbf{x}, \mathbf{y}) = (\gamma \cdot \mathbf{x}^\top \mathbf{y} + r)^d$$

- **参数:**

- γ : 缩放因子, 控制对内积的敏感度。
 - r : 偏置项。
 - d : 多项式的阶数, 决定特征的组合复杂度。

- **特点:**

- 可以处理非线性的数据。
 - 随着阶数 d 的增加, 模型复杂度增加, 可能会导致过拟合。

- **高斯径向基核 (RBF Kernel)**

- **公式:**

$$K(\mathbf{x}, \mathbf{y}) = \exp(-\gamma \|\mathbf{x} - \mathbf{y}\|^2)$$

- **参数:**

- γ : 控制高斯分布的宽度, 影响点与点之间的相似度。

- **特点:**

- 隐式地将样本映射到无穷维空间, 适用于大多数数据分布。
 - 非常灵活, 能够捕捉数据中的局部特征, 但需要合理选择 (γ) 以避免过拟合或欠拟合。

- **Sigmoid 核 (Sigmoid Kernel)**

- **公式:**

$$K(\mathbf{x}, \mathbf{y}) = \tanh(\gamma \cdot \mathbf{x}^\top \mathbf{y} + r)$$

- **特点:**

- 受神经网络启发, 类似于激活函数。
 - 不常用, 性能较其他核函数差。

Experimental Studies

以下的程序生成了一个3D的数据集。一共有1000个数据, 被分成了两大类: C0 与 C1。模型利用该数据做训练, 同时利用程序新生成与训练数据同分布的500个数据 (250个为C0类, 250个数据为C1类) 来做测试。训练集和数据集的数据分布如下:

```
def make_moons_3d(n_samples=500, noise=0.1):  
    # Generate the original 2D make_moons data  
    t = np.linspace(0, 2 * np.pi, n_samples)  
    x = 1.5 * np.cos(t)  
    y = np.sin(t)  
    z = np.sin(2 * t) # Adding a sinusoidal variation in the third dimension
```

```
# Concatenating the positive and negative moons with an offset and noise
X = np.vstack([np.column_stack([x, y, z]), np.column_stack([-x, y - 1, -z])])
y = np.hstack([np.zeros(n_samples), np.ones(n_samples)])

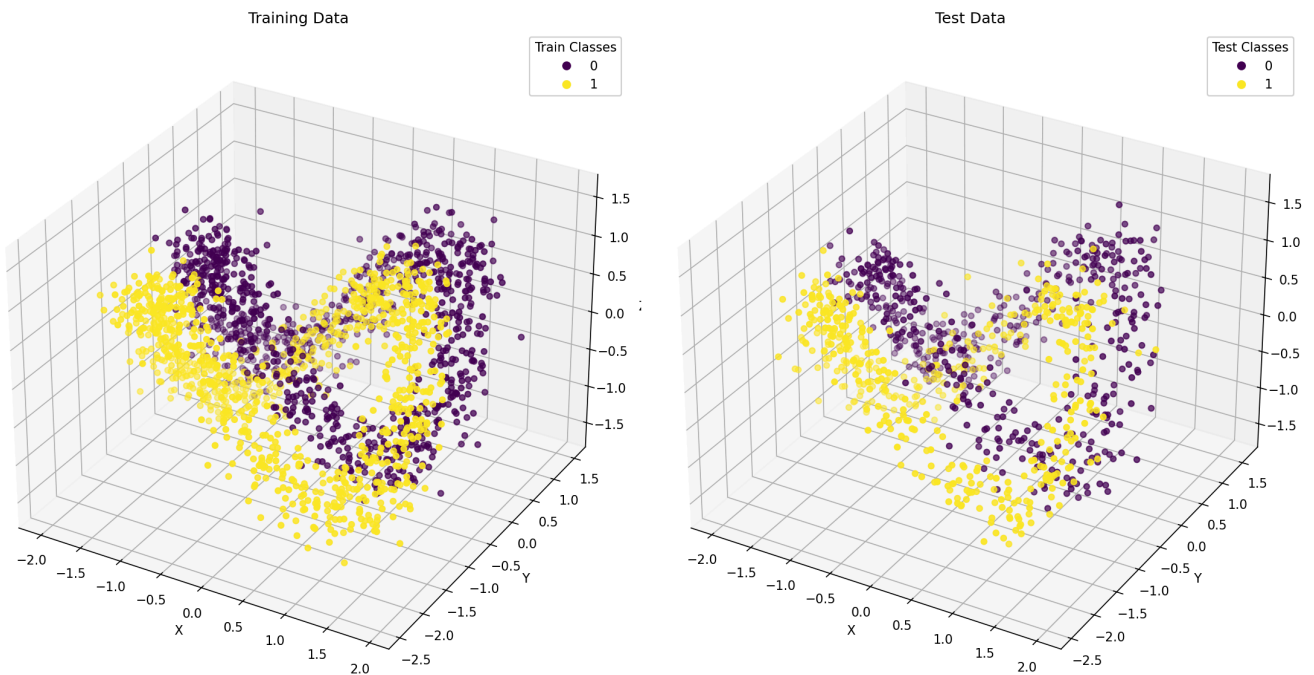
# Adding Gaussian noise
X += np.random.normal(scale=noise, size=X.shape)

return X, y
```

生成完整数据集

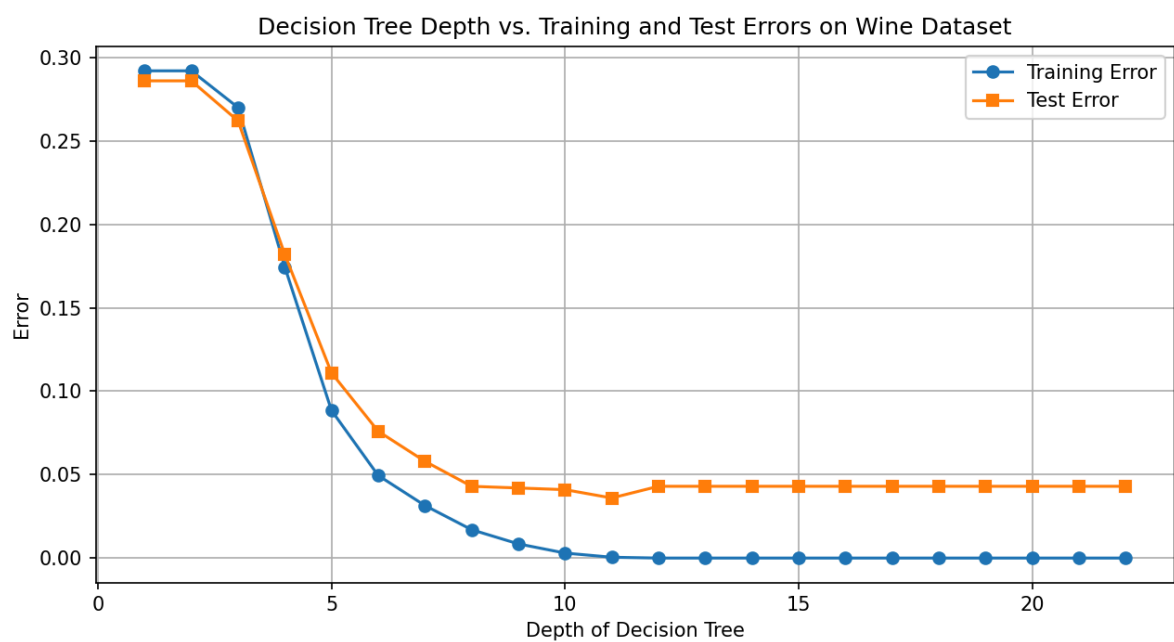
```
X, y = make_moons_3d(n_samples=1500, noise=0.2)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=1/3, random_state=42) # 使用 train_test_split 划分数据集
```



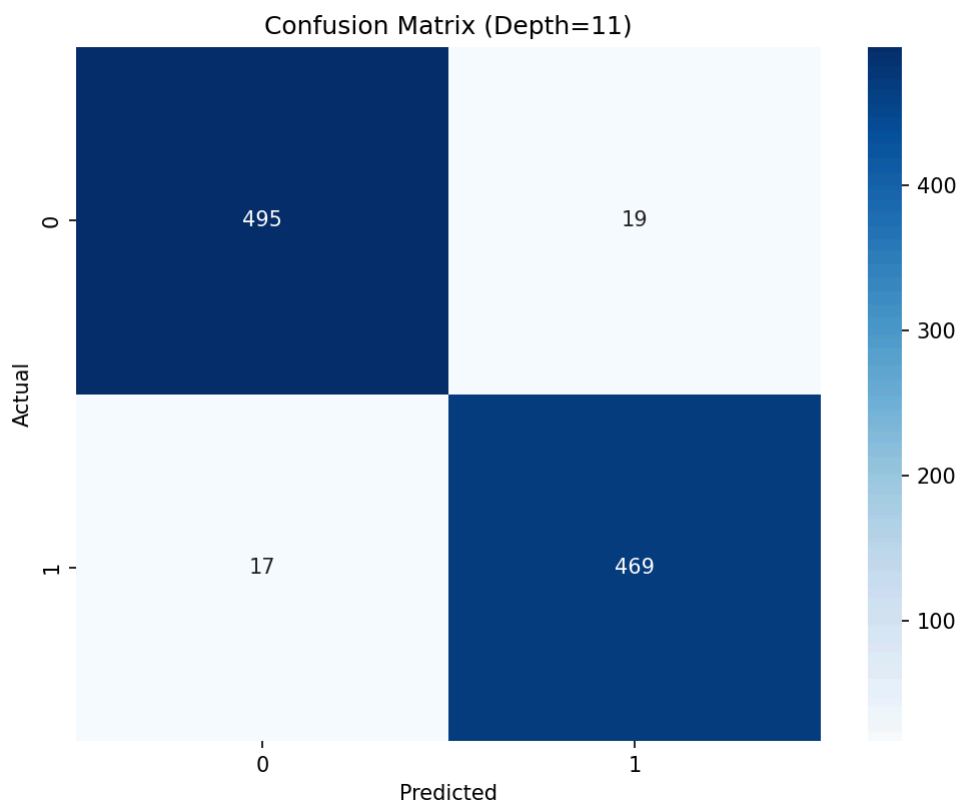
(1) Decision Trees

使用决策树进行数据分类时，决定其性能的超参数为决策树的深度，对于不同深度的决策树，其在训练集和测试集上的误差曲线如下所示：



由上可知，深度为11的决策树在测试集上的分类效果最好，故采用深度为11的决策树对数据进行分类，并对其分类性能进行分析，结果如下：

- 混淆矩阵



- 详细分类报告

	precision	recall	f1-score	support
0.0	0.97	0.96	0.96	514

	precision	recall	f1-score	support
1.0	0.96	0.97	0.96	486
accuracy			0.96	1000
macro avg	0.96	0.96	0.96	1000
weighted avg	0.96	0.96	0.96	1000

• **Decision Trees 性能指标**

准确率: 0.9640

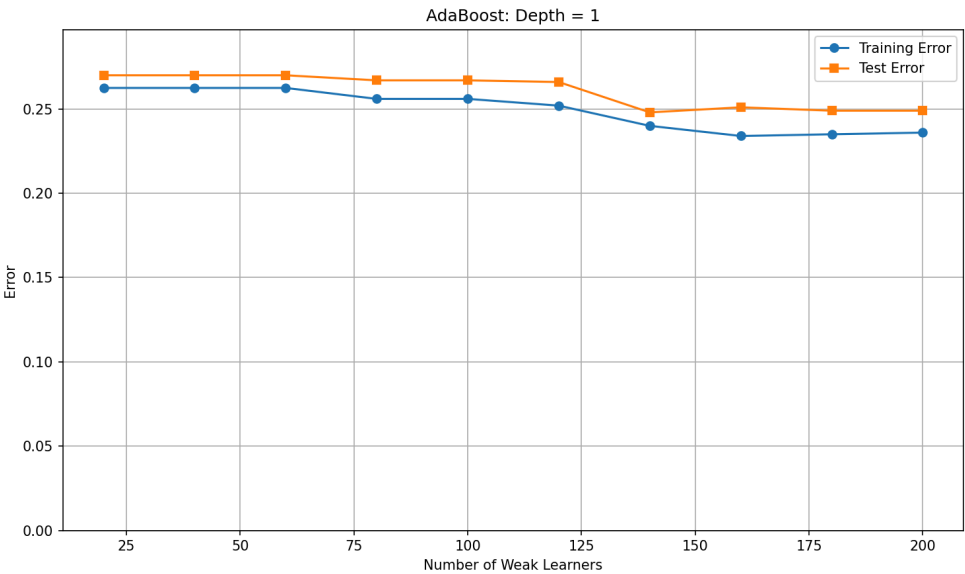
决策树是一种树状结构的监督学习算法，通过特征属性的测试进行数据分类。在本实验中，我们选择深度为11的决策树进行模型训练。决策树模型表现良好，准确率达到 96.40%。但在复杂数据集上，表现有限，主要受到树深度的影响。深度过大时可能导致过拟合，而过小则会欠拟合。

决策树的优势在于其直观性和可解释性，能快速呈现决策逻辑，适合于初步数据分析和探索。但在实际应用中，如果数据集具有较大的复杂性，决策树可能无法很好地捕捉到数据的底层模式。

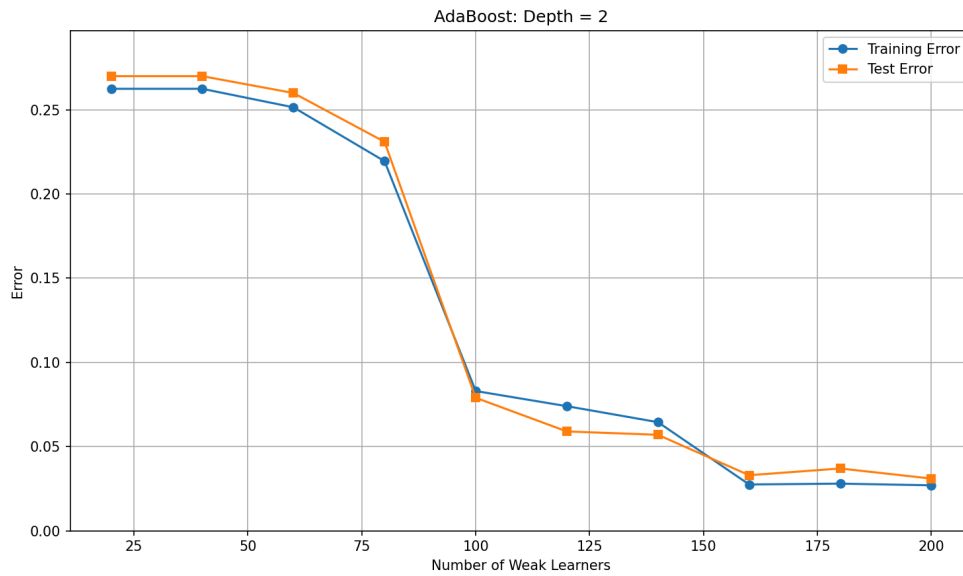
(2) **AdaBoost + Decision Trees**

使用AdaBoost + 决策树的集成学习方法进行数据分类时，决定其性能的超参数包括弱学习器的数量，以及每个弱学习器决策树的深度，对于不同数量的决策树以及不同深度的决策树，其在训练集和测试集上的误差曲线如下所示：

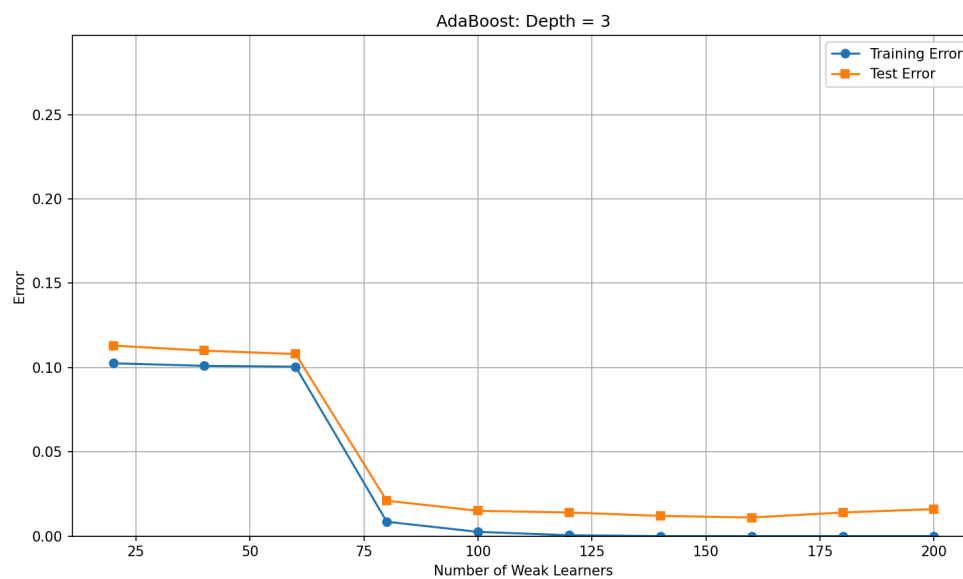
• **Depth = 1**



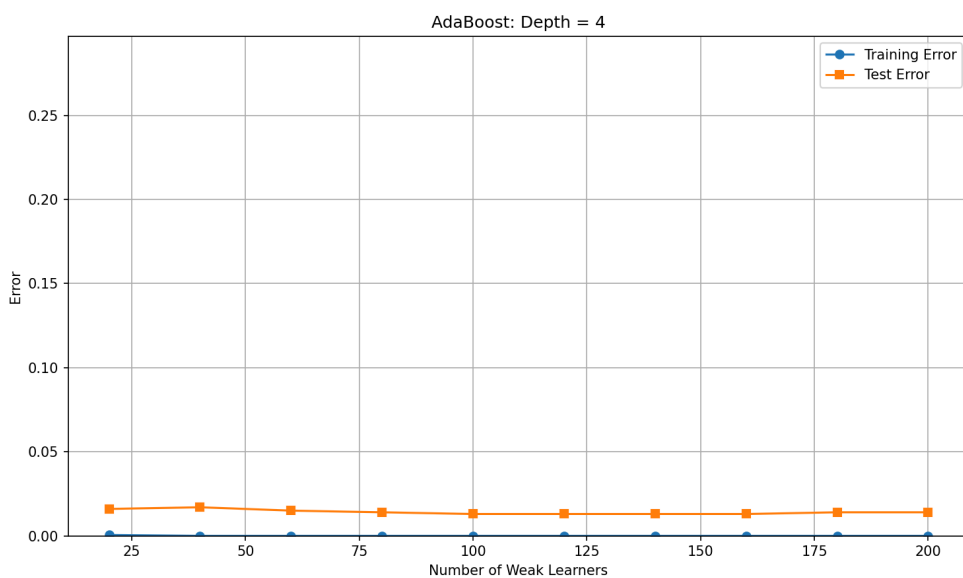
• **Depth = 2**



- Depth = 3

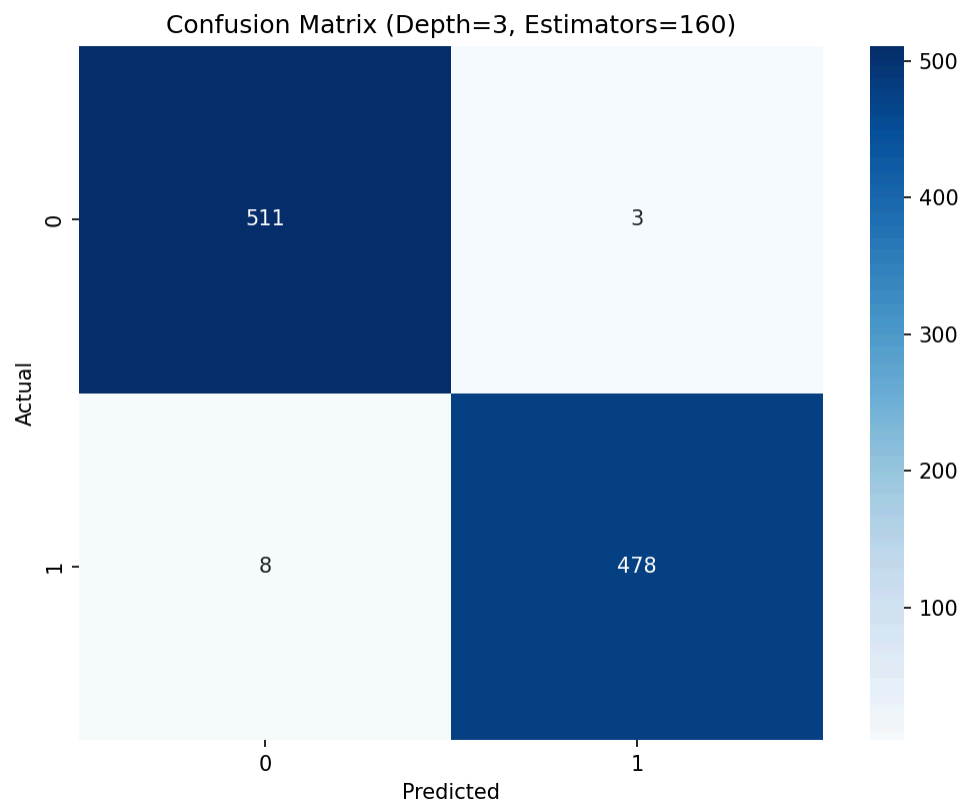


- Depth = 4



由上可知，使用深度为3的决策树作为弱学习器，使用160个弱学习器进行集成学习时，在测试集上的分类效果最好，故采用160个深度为3的决策树对数据进行分类，并对其分类性能进行分析，结果如下：

• 混淆矩阵



• 详细分类报告

	precision	recall	f1-score	support
0.0	0.98	0.99	0.99	514
1.0	0.99	0.98	0.99	486
accuracy			0.99	1000
macro avg	0.99	0.99	0.99	1000
weighted avg	0.99	0.99	0.99	1000

• AdaBoost + Decision Trees 性能指标

准确率: 0.9890

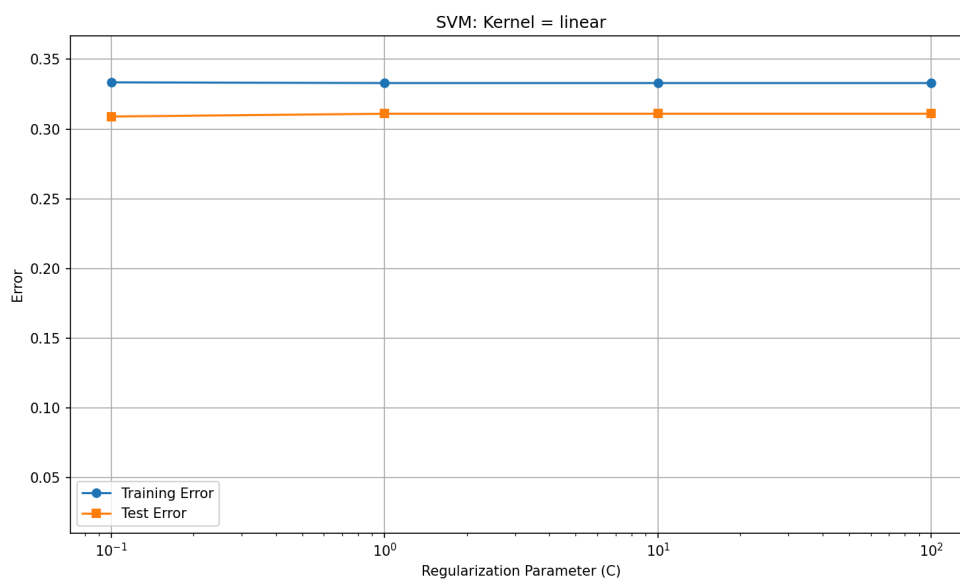
AdaBoost (Adaptive Boosting) 是一种集成学习方法，通过将多个弱学习器（例如小型决策树）组合在一起，以提高分类性能。在本实验中，使用160个深度为3的决策树作为弱学习器。AdaBoost的模型表现最佳，准确率高达98.90%，集成多个弱学习器显著提高了模型的鲁棒性，并减少了单一决策树容易出现的过拟合问题。

通过逐步调整样本权重，AdaBoost能够更有效地关注难以分类的样本，从而在处理样本不平衡或者噪声较多的问题时，能够维持良好的分类性能。这种集成学习方法在提高性能的同时，也增强了模型的稳定性，使其在新数据上的泛化能力更强。

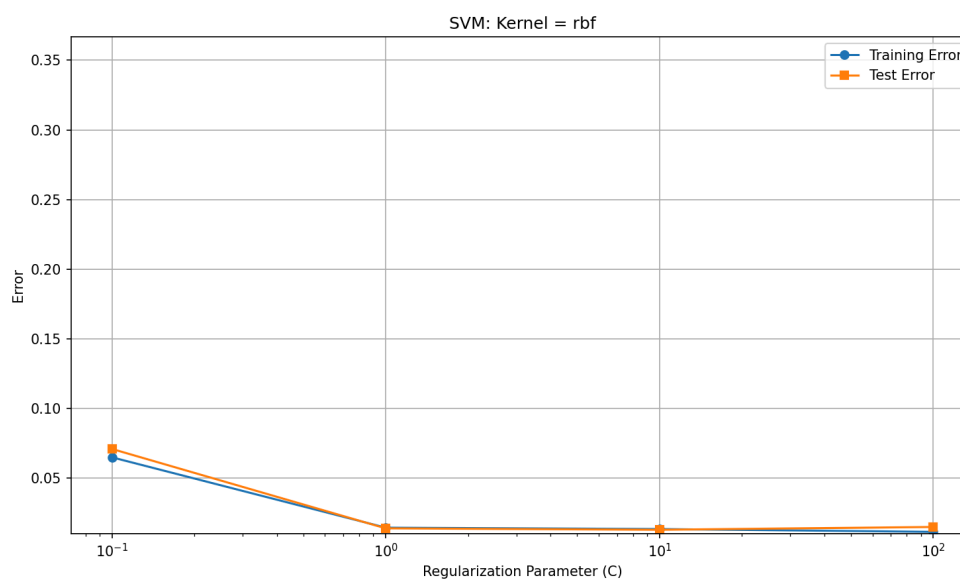
(3) SVM + Kernel Methods

使用SVM + Kernel Methods进行数据分类时，决定其性能的超参数包括正则化参数 C 和核函数的类型对于不同类型的核函数及正则化参数 C ，其在训练集和测试集上的误差曲线如下所示：

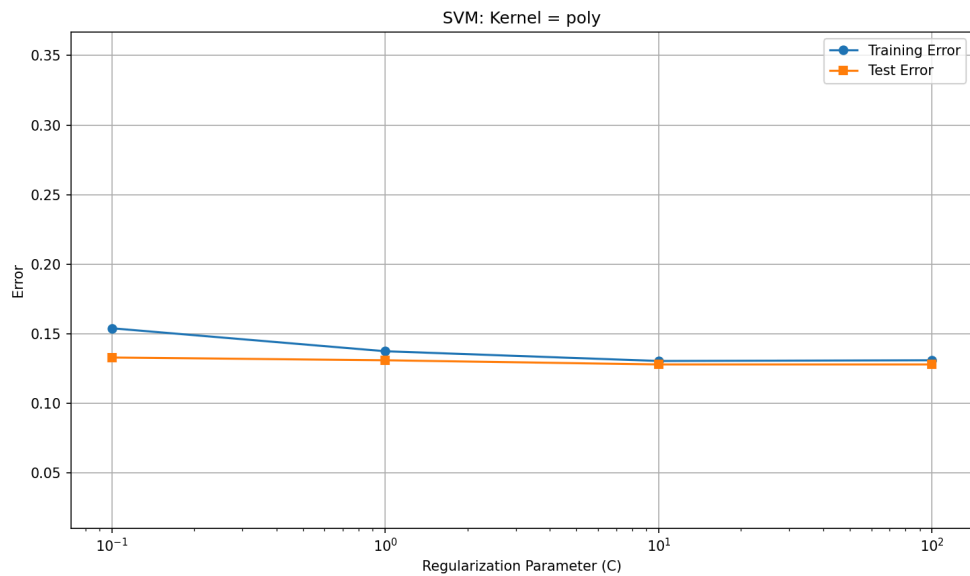
- 线性核



- 高斯径向基核

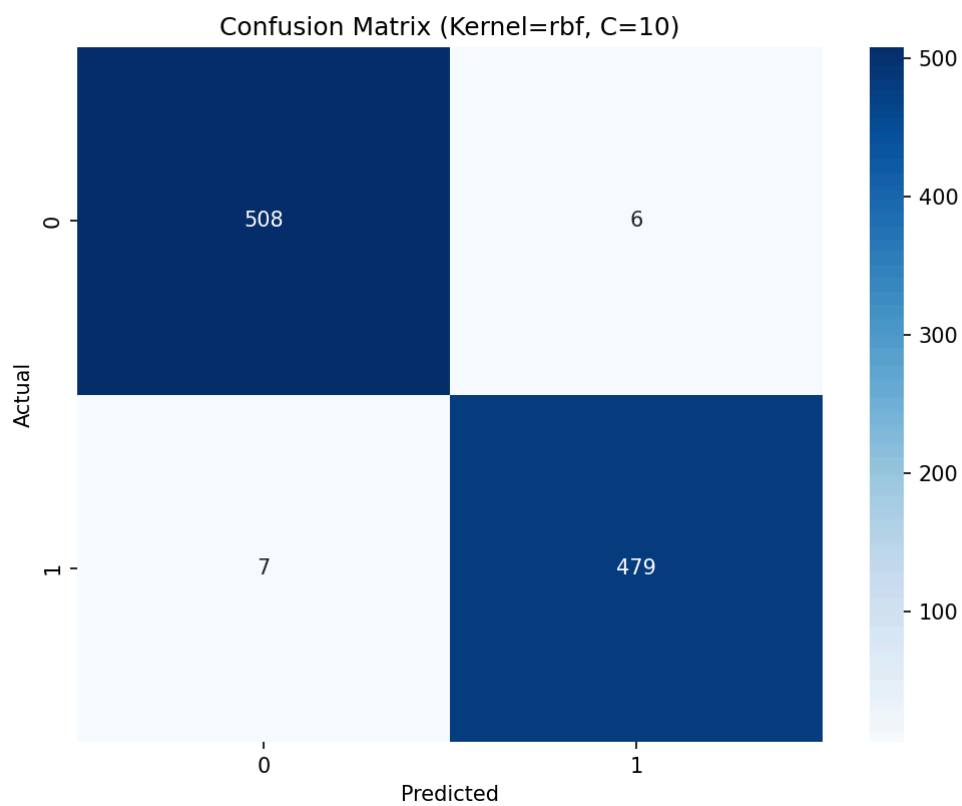


- 多项式核



由上可知，使用高斯径向基核作为核函数，正则化参数 C 设为 10 时，在测试集上的分类效果最好，故采用 C = 10，高斯径向基核的超参数对数据进行分类，并对其分类性能进行分析，结果如下：

- 混淆矩阵



- 详细分类报告

	precision	recall	f1-score	support
0.0	0.99	0.99	0.99	514
1.0	0.99	0.99	0.99	486
accuracy			0.99	1000

	precision	recall	f1-score	support
macro avg	0.99	0.99	0.99	1000
weighted avg	0.99	0.99	0.99	1000

• **SVM + Kernel Methods 性能指标**

准确率: 0.9870

支持向量机（SVM）是一种基于最大化边际的分类器，通过使用核方法能够处理非线性分类问题。在本实验中，选择高斯径向基核（RBF）和正则化参数 $C = 10$ 作为模型的超参数。SVM模型在准确率上达到98.70%，表现也相当优秀。通过高斯径向基核，SVM能够捕捉到数据的复杂特征，从而在非线性数据集上表现出色。

该模型对参数的选择较为敏感，尤其是正则化参数 C 的调整会直接影响模型的复杂性和过拟合风险。在合理选择 C 和核函数的情况下，SVM能够提供与AdaBoost相似的性能。